

Mobility Data Analyst project

Thibaut Lemichel

March 2025

1 Exercise 1

I'm running PostgreSQL 15 on the port 5433, and the password of postgres user is different, so you may have to change them if you want to follow step by step the following exercise.

1.1 Indicate the city you have chosen.

I've chosen Lille, in France.

1.2 Downloading and compressing

I've downloaded the .osm file using the following bash line.

```
1 wget -O lille.osm "http://www.overpass-api.de/api/xapi?*["bbox
  = '3.0226,50.6146,3.0975,50.6529']["@meta]"
```

Listing 1: Command ran to download lille.osm

Then to compressed it, I used the following command :

```
1 sed -E -i .org "s/version=\"[0-9]+\\"_timestamp=\"[^\"]+\\" changeset
  =\"[0-9]+\\" uid=\"[0-9]+\\" user=\"[^\"]+\\"//g" lille.osm
```

Listing 2: Command ran to compressed lille.osm

Size of the original osm file : 317.5MB.

Size of the compressed file : 182.4MB.

1.3 Modifying mapconfig.brussels.xml

I changed the mapconfig file to fit to the real french traffic rules.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <configuration>
3 <tag_name name="highway" id="1">
4   <tag_value name="motorway" id="101" priority="1.0" maxspeed="
    130"/>
5   <tag_value name="motorway_link" id="102" priority="1.0"
    maxspeed="130"/>
```

```

6   <tag_value name="motorway_junction" id="103" priority="1.0"
    maxspeed="130"/>
7   <tag_value name="trunk" id="104" priority="1.1" maxspeed="110"/
    >
8   <tag_value name="primary" id="106" priority="1.3" maxspeed="90"
    />
9   <tag_value name="secondary" id="108" priority="1.6" maxspeed="
    70"/>
10  <tag_value name="tertiary" id="110" priority="2.0" maxspeed="50
    "/>
11  <tag_value name="residential" id="112" priority="2.5" maxspeed=
    "30"/>
12  <tag_value name="service" id="115" priority="3.0" maxspeed="20"
    />
13 </tag_name>
14 </configuration>

```

Listing 3: mapconfig_lille_v1.xml

1.4 Running the osm2pgrouting tool.

1.4.1 If you installed Pg with pos

The command than I ran:

```

1 osm2pgrouting -h localhost -p 5433 -U postgres -W "toto" --f "C:\
    Users\Administrateur\Documents\MobilityDB\ProjectLille\lille.
    osm" --dbname lilleproject -c "C:\Users\Administrateur\
    Documents\MobilityDB\ProjectLille\mapconfig_lille_v1.xml"

```

Listing 4: osm2pgrouting command

1.4.2 Which of the tables contains the information that was originally provided in the config.xml file? How many tuples that this table has?

The informations from the config.xml file is contained in the table **ways**

To know how many tuples in it we can run this sql command :

```

1 SELECT COUNT(*) FROM ways;
2 SELECT COUNT(*) FROM ways_vertices_pgr;

```

Listing 5: Command to have the number of tuples

For ways we have 34491 tuples, they are representing the edges (road segments). And for the second one, it is displayed 30501 tuples, which means that we have 30501 nodes (road intersections).

1.5 Running the osm2pgsql tool.

1.5.1 Downloading osm2pgsql tool.

1.5.2 Run the command

```
1 osm2pgsql -c -H localhost -P 5433 -U postgres -W -d lilleproject --  
   proj=3857 "C:\Users\Administrateur\Documents\MobilityDB\  
   ProjectLille\lille.osm" --style "C:\Program Files\osm2pgsql-  
   latest-x64\osm2pgsql-bin\default.style"
```

Listing 6: Command ran

I had to add the default.style in the commande because it wasn't found in the initial command.

The following tables have been created:

- planet_osm_point
- planet_osm_line
- planet_osm_polygon
- planet_osm_roads

1.5.3 How many tuples contains the planet* tables?

By running this sql command :

```
1 SELECT 'planet_osm_line' AS table_name, COUNT(*) AS row_count FROM  
   planet_osm_line  
2 UNION ALL  
3 SELECT 'planet_osm_point', COUNT(*) FROM planet_osm_point  
4 UNION ALL  
5 SELECT 'planet_osm_polygon', COUNT(*) FROM planet_osm_polygon  
6 UNION ALL  
7 SELECT 'planet_osm_roads', COUNT(*) FROM planet_osm_roads;
```

Listing 7: Command to have the number of rows in each planet* table

	table_name	row_count
1	planet_osm_roads	3855
2	planet_osm_line	33485
3	planet_osm_point	111694
4	planet_osm_poly...	148845

Figure 1: Number of rows for each planet* table

As we can see on the screenshot (Figure 1), we have :

- planet_osm_roads, representing roads network, has 3855 rows.
- planet_osm_line, representing streams, fences, has 33485 rows.
- planet_osm_polygon, representing building, parks, has 111694 rows.
- planet_osm_point, representing shops, cafe, has 148845 rows.

1.6 The lille_preparedata.sql file

As my data set is Lille, I found some information on this website <https://www.bien-dans-ma-ville.fr/lille-59350/quartier-lille-sud/>. Informations about population and density of population are the true ones. However I couldn't find the rest of the data so I've just randomly given them values.

```

1 DROP TABLE IF EXISTS Municipalities;
2 CREATE TABLE Municipalities(MunicipalityId int PRIMARY KEY,
3   MunicipalityName text, Population int, PercPop float,
4   PopDensityKm2 int,
5   NoEnterp int, PercEnterp float);
6 INSERT INTO Municipalities VALUES
7 (1, 'Lille-Moulins ', 19991, 0.12, 11168, 1230, 0.05),
8 (2, 'Wazemmes ', 27515, 0.09, 16476, 980, 0.04),
9 (3, 'Lille-Centre ', 28917, 0.06, 9512, 620, 0.03),
10 (4, 'Fives ', 21025, 0.09, 6893, 850, 0.04),
11 (5, 'Vieux-Lille ', 18772, 0.08, 10147, 1000, 0.05),
12 (6, 'Saint-Maurice Pellevoisin ', 17031, 0.09, 8779, 990, 0.06),
13 (7, 'Faubourg de B thune ', 7621, 0.10, 7056, 750, 0.02),
14 (8, 'Lille-Sud ', 20916, 0.12, 6063, 650, 0.05),
15 (9, 'Bois Blancs ', 8951, 0.08, 5115, 1180, 0.03),
16 (10, 'Vauban-Esquermes ', 18772, 0.07, 7954, 870, 0.04);

```

Listing 8: lille_preparedata.sql

1.7 Run the berlinmod_datagenerator.sql script

1.8 Run the berlinmod_datagenerator.sql PSM. Indicate any parameter you use (scale_factor, etc).

I used factor(0.007).

```

1 INFO:
2 -----
3 INFO: Starting the BerlinMOD data generator with scale factor
4 0.007
5 INFO:
6 -----
7 INFO: Parameters:
8 INFO: -----
9 INFO: No. of vehicles = 167, No. of days = 4, Start day =
10 2020-06-01
11 INFO: Path mode = Fastest Path, Disturb data = f

```

```

8 INFO: Verbosity = minimal, Trip generation = C, Index type = GiST
9 INFO: Execution started at 2025-04-10 15:11:45.447589+02
10 INFO:
-----

11 INFO: Creating the Destinations table
12 NOTICE: la table destinations n'existe pas, poursuite du
    traitement
13 INFO: Creating the VehicleNodes, Vehicles, and Neighbourhoods
    tables
14 NOTICE: la table vehiclenodes n'existe pas, poursuite du
    traitement
15 NOTICE: la table vehicles n'existe pas, poursuite du
    traitement
16 NOTICE: la table neighbourhoods n'existe pas, poursuite du
    traitement
17 INFO: Creating the Licences table
18 NOTICE: la table licences n'existe pas, poursuite du
    traitement
19 INFO: Creating the Points and Regions tables
20 NOTICE: la table points n'existe pas, poursuite du
    traitement
21 NOTICE: Creating indexes on table Points
22 NOTICE: la table regions n'existe pas, poursuite du
    traitement
23 NOTICE: Creating indexes on table Regions
24 INFO: Creating the Instants and Periods tables
25 NOTICE: la table instants n'existe pas, poursuite du
    traitement
26 NOTICE: la table periods n'existe pas, poursuite du
    traitement
27 NOTICE: Creating indexes on table Periods
28 INFO: Creating the LeisureTrips table
29 NOTICE: la table leisuretrips n'existe pas, poursuite du
    traitement
30 INFO: Creating the Paths table
31 NOTICE: la table paths n'existe pas, poursuite du traitement
32 INFO: Call to pgRouting to compute 902 paths
33 INFO: Creation of the Trips table
34 NOTICE: la table trips n'existe pas, poursuite du traitement
35 INFO: Vehicles 1 to 100
36 INFO: Vehicles 101 to 167
37 NOTICE: Creating indexes on table Trips
38 INFO:
-----

39 INFO: Execution started at 2025-04-10 15:11:45.447589+02
40 INFO: Execution finished at 2025-04-10 15:12:08.031812+02
41 INFO: Execution time 00:00:22.584223
42 INFO: Call to pgRouting with 902 paths lasted 00:00:02.268081
43 INFO: Number of trips generated 1904
44 INFO:
-----

45 berlinmod_datagenerator
46 -----
47 THE END

```

```
48 (1 row)
```

Listing 9: Return

1.9 Export the database (dump)

```
1 pg_dump -U postgres -h localhost -p 5433 -F p -d lilleproject -f
  lilleProject.dump
```

Listing 10: Export the database

2 Exercice 2

You can see the different sql query and screenshots of the different results displayed in the pdf, however I know that is not the most convenient to copy paste due to the numbers on the left size, that is why you can find the sql query in .sql file for each question.

- 2.1 For each vehicle write an SQL query to find the maximum length (in Km) over all its trips. The result should show three columns: vehicleId, time duration of the trip and the trajectory of this trip. Take into consideration that if a vehicle has multiple trips with the same maximum length, it must appear multiple times in the result set (one for each duration time).**

```
1 select t1.vehicleId,
2       duration(t1.trip) as duration,
3       t1.trajectory
4 from trips t1
5 join (
6       select vehicleId, MAX(length(trip)) as max_length
7       from trips
8       group by vehicleId
9 ) t2 on t1.vehicleId = t2.vehicleId and length(t1.trip) = t2.
    max_length;
```

Listing 11: Code question 1

	vehicleid	duration	trajectory
1	2	00:23:42.028992	0102000020110...
2	3	00:46:49.737151	0102000020110...
3	1	00:26:11.698347	0102000020110...
4	4	01:49:07.963697	0102000020110...
5	23	01:55:25.910685	0102000020110...
6	24	02:24:02.524924	0102000020110...
7	8	02:01:52.080588	0102000020110...
8	5	02:01:42.455962	0102000020110...
9	15	02:01:29.632637	0102000020110...
10	6	00:32:30.647446	0102000020110...
11	7	02:04:45.262474	0102000020110...

Figure 2: Question 1

2.2 Write an SQL query that computes the trips that cross at least two adjacent municipalities. You should only show the tripid and trips restricted by these boundaries and the time when this intersection occurred. Limit the result to the first 30 tuples, showing first those with less starting time of the trip. Also display the result in QGIS.

```

1 SELECT
2   t.tripid,
3   atgeometry(t.trip, ST_Union(m1.municipalitygeo, m2.
4     municipalitygeo)) AS restricted_trip,
5   timestamps(atgeometry(t.trip, ST_Union(m1.municipalitygeo, m2.
6     municipalitygeo))) AS intersection_time
7 FROM
8   trips t
9 JOIN municipalities m1 ON eintersects(t.trip, m1.municipalitygeo)
10 JOIN municipalities m2 ON eintersects(t.trip, m2.municipalitygeo)
11 WHERE
12   m1.municipalityid < m2.municipalityid
13   AND ST_Touches(m1.municipalitygeo, m2.municipalitygeo)
14 ORDER BY
15   startTimestamp(t.trip)
16 LIMIT 30;

```

Listing 12: Code question 2

	tripid	restricted_trip	intersection_time
1	766	{[010100002011... 08:03:03.47795...	{[2020-06-01 08:03:03.47795...
2	766	{[010100002011... 08:06:21.23295...	{[2020-06-01 08:06:21.23295...
3	766	{[010100002011... 08:03:03.47795...	{[2020-06-01 08:03:03.47795...
4	1473	{[010100002011... 08:18:37.97593...	{[2020-06-01 08:18:37.97593...
5	1473	{[010100002011... 08:21:37.83785...	{[2020-06-01 08:21:37.83785...
6	1473	{[010100002011... 08:13:36.69245...	{[2020-06-01 08:13:36.69245...
7	1473	{[010100002011... 08:18:37.97593...	{[2020-06-01 08:18:37.97593...
8	524	{[010100002011... 09:22:38.49508...	{[2020-06-01 09:22:38.49508...
9	524	{[010100002011... 09:33:48.20530...	{[2020-06-01 09:33:48.20530...
10	524	{[010100002011... 09:30:25.90471...	{[2020-06-01 09:30:25.90471...
11	524	{[010100002011... 09:30:25.90471...	{[2020-06-01 09:30:25.90471...

Figure 3: Question 2

2.3 Write an SQL query to compute when and where a pair of vehicles have been at 50 m or less from each other. If necessary, display only the first 20 pairs of vehicles that satisfy this condition.

```

1 SELECT
2   t1.vehicleid AS vehicle1,
3   t2.vehicleid AS vehicle2,
4   tdwithin(t1.trip, t2.trip, 50.0::float8, TRUE) AS proximity_times
5   ,
6   nearestApproachInstant(t1.trip, t2.trip) AS closest_moment,
7   getValue(nearestApproachInstant(t1.trip, t2.trip)) AS
8     closest_point,
9   nearestApproachDistance(t1.trip, t2.trip) AS min_distance
10 FROM
11   trips t1
12 JOIN trips t2
13   ON t1.vehicleid < t2.vehicleid
14 WHERE
15   tdwithin(t1.trip, t2.trip, 50.0::float8, TRUE) IS NOT NULL
16 LIMIT 20;

```

Listing 13: Code question 3

	vehicle1	vehicle2	proximity_times	closest_moment	closest_point	min_distance
1	1	25	([t@2020-06-01 09:57:42.47978...	0101000020110... 09:57:57.95101...	0101000020110...	0
2	1	42	([t@2020-06-01 10:05:56.67355...	0101000020110... 10:07:03.27901...	0101000020110...	0
3	1	155	([t@2020-06-01 10:01:41.83660...	0101000020110... 10:01:43.88320...	0101000020110...	0
4	1	47	([t@2020-06-01 17:38:35.97935...	0101000020110... 17:40:03.00453...	0101000020110...	0
5	1	96	([t@2020-06-01 17:36:39.11105...	0101000020110... 17:36:41.53629...	0101000020110...	38,50122167058...
6	1	13	([t@2020-06-02 17:25:45.18147...	0101000020110... 17:27:20.04153...	0101000020110...	9
7	1	33	([t@2020-06-02 17:28:15.76609...	0101000020110... 17:28:20.24406...	0101000020110...	0
8	1	43	([t@2020-06-02 21:26:24.10575...	0101000020110... 21:26:28.83649...	0101000020110...	25,39035369306...
9	2	36	([t@2020-06-01 09:39:04.15352...	0101000020110... 09:39:15.62471...	0101000020110...	0
10	2	144	([t@2020-06-01 09:37:15.92684...	0101000020110... 09:37:20.76402...	0101000020110...	38,67854646777...
11	2	91	([t@2020-06-01 17:16:21.81960...	0101000020110... 17:16:26.88725...	0101000020110...	15,41215656323...

Figure 4: Question 3

2.4 Write an SQL query that computes the interval [t1, t2] where t1 and t2 are the starting and ending times of the whole period generated with the berlinmod.generate method. Also compute t, the middle point of this interval.

We want to classify trips according to the following criteria:

- "the first half of trips": those ones that occurred completely inside the interval [t1, t)
- "the second half of trips": those ones that occurred completely inside the interval [t, t2]
- "trips that crossed t": those ones that started in [t1, t) and finished in [t, t2].

Propose a query that returns only just one tuple with three columns indicating how many trips belong to each of the categories above

```

1 WITH time_bounds AS (
2   SELECT
3     MIN(startTimestamp(trip)) AS t1,
4     MAX(endTimestamp(trip)) AS t2
5   FROM trips
6 ),
7 middle_point AS (
8   SELECT
9     t1,
10    t2,
11    t1 + ((t2 - t1) / 2) AS t
12  FROM time_bounds
13 ),
14 classified AS (
15   SELECT
16     CASE
17       WHEN startTimestamp(trip) >= t1 AND endTimestamp(trip) < t
18       THEN 'the_first_half_of_trips'
19       WHEN startTimestamp(trip) >= t AND endTimestamp(trip) <= t2
20       THEN 'the_second_half_of_trips'
21     END AS category
22   FROM trips
23 )
24 SELECT
25   SUM(CASE WHEN category = 'the_first_half_of_trips' THEN 1 ELSE 0 END) AS first_half,
26   SUM(CASE WHEN category = 'the_second_half_of_trips' THEN 1 ELSE 0 END) AS second_half,
27   SUM(CASE WHEN category = 'trips that crossed t' THEN 1 ELSE 0 END) AS crossed_t
28 FROM classified

```

```

19     WHEN startTimestamp(trip) < t AND endTimestamp(trip) >= t
20     THEN 'trips_that_crossed_t'
21     END AS category
22 FROM trips, middle_point
23 )
24 SELECT
25     COUNT(*) FILTER (WHERE category = 'the_first_half_of_trips') AS
26     first_half,
27     COUNT(*) FILTER (WHERE category = 'the_second_half_of_trips') AS
28     second_half,
29     COUNT(*) FILTER (WHERE category = 'trips_that_crossed_t') AS
30     crossing
31 FROM classified;

```

Listing 14: Code question 4

	first_half	second_half	crossing
1	929	975	0

Figure 5: Question 4

2.5 We want to calculate the usage of edges by trips, i.e., to compute how many trips traversed each of the edges on the network, and create a table called “HeatMap” with the result. Write an SQL query to create and populate such table.

```

1 DROP TABLE IF EXISTS HeatMap;
2
3 CREATE TABLE HeatMap (
4     edge_id INTEGER,
5     trip_count INTEGER,
6     geom geometry(LineString, 4326)
7 );
8
9 WITH transformed_trips AS (
10     SELECT
11         tripid,
12         ST_Transform(trajecory(trip), 4326) AS traj_geom
13     FROM trips
14     WHERE trip IS NOT NULL
15 )
16 INSERT INTO HeatMap (edge_id, trip_count, geom)
17 SELECT
18     w.gid AS edge_id,
19     COUNT(DISTINCT t.tripid) AS trip_count,
20     w.the_geom
21 FROM
22     ways w
23 JOIN transformed_trips t
24     ON w.the_geom && t.traj_geom

```

```

25 AND ST_Intersects(w.the_geom, t.traj_geom)
26 GROUP BY
27     w.gid, w.the_geom;

```

Listing 15: Code question 5

To make the request faster I've added limits. I actually used the following code :

```

1 DROP TABLE IF EXISTS HeatMap;
2
3 CREATE TABLE HeatMap (
4     edge_id    INTEGER,
5     trip_count INTEGER,
6     geom       geometry(LineString, 4326)
7 );
8
9 WITH transformed_trips AS (
10     SELECT
11         t.tripid,
12         ST_Transform(trajecory(t.trip), 4326) AS traj_geom
13     FROM trips t
14     WHERE t.trip IS NOT NULL
15     limit 500
16 )
17 INSERT INTO HeatMap (edge_id, trip_count, geom)
18 SELECT
19     w.gid AS edge_id,
20     COUNT(DISTINCT t.tripid) AS trip_count,
21     w.the_geom
22 FROM ways w
23 JOIN transformed_trips t
24     ON w.the_geom && t.traj_geom
25 AND ST_Intersects(w.the_geom, t.traj_geom)
26 GROUP BY w.gid, w.the_geom
27 LIMIT 1000;

```

Listing 16: question 5 with limits

And then to visualize the table of heatmap, I just used the following query:

```

1 SELECT * FROM heatmap;

```

Listing 17: Query to visualize heatmap

And this is the return table (Figure 6).


```

10      -- Work to home
11      WHEN ST_Intersects(ST_StartPoint(trajecotory(t.trip)), wr.geom
12  )
13      AND ST_Intersects(ST_EndPoint(trajecotory(t.trip)), hr.geom)
14      THEN 'work_home'
15
16      -- Leisure weekday
17      WHEN lt.vehicleid IS NOT NULL AND EXTRACT(DOW FROM t.
18  startdate) BETWEEN 1 AND 5
19      THEN 'leisure_weekday'
20
21      -- Leisure weekend
22      WHEN lt.vehicleid IS NOT NULL AND EXTRACT(DOW FROM t.
23  startdate) IN (0, 6)
24      THEN 'leisure_weekend'
25  END AS trip_type,
26  endTimestamp(t.trip) - startTimestamp(t.trip) AS duration
27  FROM trips t
28  LEFT JOIN LeisureTrips lt
29  ON t.vehicleid = lt.vehicleid AND t.startdate = lt.startdate
30  AND t.seqno = lt.seqno
31  LEFT JOIN HomeRegions hr
32  ON hr.id = t.vehicleid
33  LEFT JOIN WorkRegions wr
34  ON wr.id = t.vehicleid
35  WHERE t.trip IS NOT NULL
36  )
37  SELECT
38  trip_type,
39  COUNT(*) AS total_trips,
40  MIN(duration) AS min_duration,
41  MAX(duration) AS max_duration,
42  AVG(duration) AS avg_duration
43  FROM classified_trips
44  WHERE trip_type IS NOT NULL
45  GROUP BY trip_type;

```

Listing 18: code question 7 a

	trip_type	total_trips	min_duration	max_duration	avg_duration
1	home_work	4	00:01:37.235224	00:04:13.328452	00:03:07.266782
2	leisure_weekday	568	00:00:36.38147	03:06:04.737468	00:56:18.041776

Figure 8: Question 7 a)

2.7.2 Display in QGIS the different kinds of trips.

To visualize it better I changed the code to have all trips seperated, and I created a View but I couldn't manage to have the spacial data of the view working.

```

1  DROP VIEW IF EXISTS classified_trips_view;
2
3  CREATE VIEW classified_trips_view AS
4  WITH classified_trips AS (

```

```

5 SELECT
6     t.tripid,
7     t.vehicleid,
8     -- Convert the trip trajectory (which may be in SRID 3857) into
        a geometry in SRID 4326.
9     ST_Transform(trajectory(t.trip), 4326)::geometry(LineString
        ,4326) AS geom,
10    (endTimeStamp(t.trip) - startTimeStamp(t.trip)) AS duration,
11    CASE
12        -- Home to work: trip starts in home region and ends in work
        region
13        WHEN ST_Intersects(ST_StartPoint(trajectory(t.trip)), hr.geom
        )
14            AND ST_Intersects(ST_EndPoint(trajectory(t.trip)), wr.
        geom)
15        THEN 'home_work'
16        -- Work to home: trip starts in work region and ends in home
        region
17        WHEN ST_Intersects(ST_StartPoint(trajectory(t.trip)), wr.geom
        )
18            AND ST_Intersects(ST_EndPoint(trajectory(t.trip)), hr.
        geom)
19        THEN 'work_home'
20        -- Leisure weekday: trip appears in LeisureTrips and the day-
        of-week is between 1 and 5
21        WHEN lt.vehicleid IS NOT NULL
22            AND EXTRACT(DOW FROM t.startdate) BETWEEN 1 AND 5
23        THEN 'leisure_weekday'
24        -- Leisure weekend: trip appears in LeisureTrips and the day-
        of-week is 0 or 6
25        WHEN lt.vehicleid IS NOT NULL
26            AND EXTRACT(DOW FROM t.startdate) IN (0, 6)
27        THEN 'leisure_weekend'
28        ELSE 'unknown'
29    END AS trip_type
30 FROM trips t
31 LEFT JOIN LeisureTrips lt
32     ON t.vehicleid = lt.vehicleid
33     AND t.startdate = lt.startdate
34     AND t.seqno = lt.seqno
35 LEFT JOIN HomeRegions hr
36     ON hr.id = t.vehicleid
37 LEFT JOIN WorkRegions wr
38     ON wr.id = t.vehicleid
39 WHERE t.trip IS NOT NULL
40 )
41 SELECT *
42 FROM classified_trips
43 WHERE trip_type <> 'unknown';

```

Listing 19: Question 7 b

2.8 Compute and visualize in QGIS the longest trip.

```

1 SELECT

```

```

2    t.tripid,
3    ST_Transform(trajecory(t.trip), 4326) AS geom,  -- convert to
      WGS84
4    ST_Length(ST_Transform(trajecory(t.trip), 4326)::geography) AS
      length_meters
5 FROM trips t
6 WHERE t.trip IS NOT NULL
7 ORDER BY length_meters DESC
8 LIMIT 1;

```

Listing 20: code question 8

	tripid	geom	length_meters
1	593	0102000020E61...	52424,94037193...

Figure 9: Question 8

2.9 Write an SQL query to compute the vehicles that have crossed, in any trip, a region in table Regions during a period in table Periods. List three identifiers: VehicleId, periodID and RegionId. (Consider using the && operator to speed up the queries).

```

1 SELECT DISTINCT
2     t.vehicleid,
3     p.periodid,
4     r.regionid
5 FROM
6     trips t
7 JOIN regions r
8     ON trajectory(t.trip) && r.geom
9     AND eintersects(t.trip, r.geom)
10 JOIN periods p
11     ON getTime(t.trip) && p.period
12 WHERE t.trip IS NOT NULL;

```

Listing 21: code question 9

2.10 List the first time at which a vehicle visited a point in table Points.

```

1 SELECT DISTINCT ON (t.vehicleid, p.pointid)
2     t.vehicleid,
3     p.pointid,
4     getTimestamp(i.inst) AS instance
5 FROM
6     trips t
7 JOIN points p

```

	vehideid	periodid	regionid
1	1	1	1
2	1	1	32
3	1	1	42
4	1	2	1
5	1	2	2

Figure 10: Question 9

```

8  ON eintersects(t.trip, p.geom)
9  CROSS JOIN LATERAL (
10   SELECT unnest(instants(atGeometry(t.trip, p.geom))) AS inst
11 ) AS i
12 WHERE getTimestamp(i.inst) IS NOT NULL
13 ORDER BY t.vehicleid, p.pointid, instance;

```

Listing 22: code question 10

	vehideid	pointid	instance
1	2	62	2020-06-01T17:...
2	2	69	2020-06-01T17:...
3	4	53	2020-06-01T18:...
4	4	63	2020-06-01T20:...
5	5	53	2020-06-01T16:...

Figure 11: Question 10

2.11 Write an SQL query that computes the number of vehicles that were active at each period in table Periods?

```

1  WITH vehicle_counts AS (
2    SELECT
3      p.periodid,
4      p.period,
5      (
6        SELECT COUNT(DISTINCT t.vehicleid)
7        FROM trips t
8        WHERE getTime(t.trip) && p.period
9      ) AS count_value
10   FROM periods p
11 )
12 SELECT
13   periodid AS regionid,

```



```

14 (
15     '[' ||
16     count_value::text || '@' || lower(period)::text || ',' ||
17     count_value::text || '@' || upper(period)::text ||
18     ']'
19 )::tint AS wcount
20 FROM vehicle_counts;

```

Listing 23: Code question 11

	regionid	wcount
1	1	[167@2020-06-02 03:01:38.27101...
2	2	[167@2020-06-04 13:49:33.50019...
3	3	[11@2020-06-02 00:04:28.43874...
4	4	[46@2020-06-02 10:35:56.97113...
5	5	[167@2020-06-02 16:12:16.98687...

Figure 12: Question 11

2.12 Write an SQL query that computes the number of trips that were active during each hour in Jun 03, 2020?

```

1 WITH hours AS (
2     SELECT
3         ts AS start_time,
4         ts + interval '1_hour' AS end_time,
5         (
6             '[' || ts::text || ',' || (ts + interval '1_hour')::text || '
7         )::tstzspan AS period
8     FROM generate_series(
9         '2020-06-03_00:00:00+00'::timestampz,
10        '2020-06-03_23:00:00+00'::timestampz,
11        interval '1_hour'
12    ) AS ts
13 )
14 SELECT
15     h.period,
16     (
17         SELECT COUNT(*)
18         FROM trips t
19         WHERE getTime(t.trip) && h.period
20     ) AS count
21 FROM hours h

```

```
22 ORDER BY h.period;
```

Listing 24: Code question 12

	period	count
1	[2020-06-03 02:00:00+02, ...	1
2	[2020-06-03 03:00:00+02, ...	0
3	[2020-06-03 04:00:00+02, ...	0
4	[2020-06-03 05:00:00+02, ...	0
5	[2020-06-03 06:00:00+02, ...	0
6	[2020-06-03 07:00:00+02, ...	0
7	[2020-06-03 08:00:00+02, ...	87

Figure 13: Question 12

2.13 Write an SQL query that computes the overall traveled distances for each vehicle per period, during the periods in table Periods, in descending order.

```
1 WITH trip_period AS (
2   SELECT
3     t.vehicleid,
4     p.periodid,
5     p.period,
6     startTimestamp(t.trip) AS trip_start,
7     endTimestamp(t.trip) AS trip_end,
8     -- Determine overlapping interval between the trip and the
9     -- period:
10    GREATEST(startTimestamp(t.trip), lower(p.period)) AS
11    overlap_start,
12    LEAST(endTimestamp(t.trip), upper(p.period)) AS overlap_end,
13    -- Compute total trip distance in meters: reproject the
14    -- trajectory to 4326 before computing geography distance
15    ST_Length(ST_Transform(trajectory(t.trip), 4326)::geography) AS
16    total_distance,
17    -- Compute durations in seconds:
18    EXTRACT(epoch FROM (endTimestamp(t.trip) - startTimestamp(t.
19    trip))) AS trip_duration,
20    EXTRACT(epoch FROM (LEAST(endTimestamp(t.trip), upper(p.period)
21    )
22    - GREATEST(startTimestamp(t.trip), lower(p.
23    period)))) AS overlap_duration
```

```

17 FROM trips t
18 JOIN periods p
19     ON getTime(t.trip) && p.period
20 )
21 SELECT
22     vehicleid,
23     periodid,
24     period,
25     SUM(total_distance * (overlap_duration / NULLIF(trip_duration, 0)
26         )) AS distance
27 FROM trip_period
28 GROUP BY vehicleid, periodid, period
29 ORDER BY distance DESC;

```

Listing 25: Code question 13

	vehicleid	periodid	period	distance
1	52	61	[2020-06-01 04:01:42.12274...	295434,0089957...
2	52	70	[2020-06-01 07:36:25.81491...	295434,0089957...
3	55	61	[2020-06-01 04:01:42.12274...	284539,7935343...
4	55	70	[2020-06-01 07:36:25.81491...	284539,7935343...
5	52	80	[2020-06-01 05:36:22.31467...	276793,3901317...
6	55	80	[2020-06-01 05:36:22.31467...	265740,7063623...
7	22	91	[2020-06-03 07:14:59.81716...	262667,9397882...

Figure 14: Question 13